



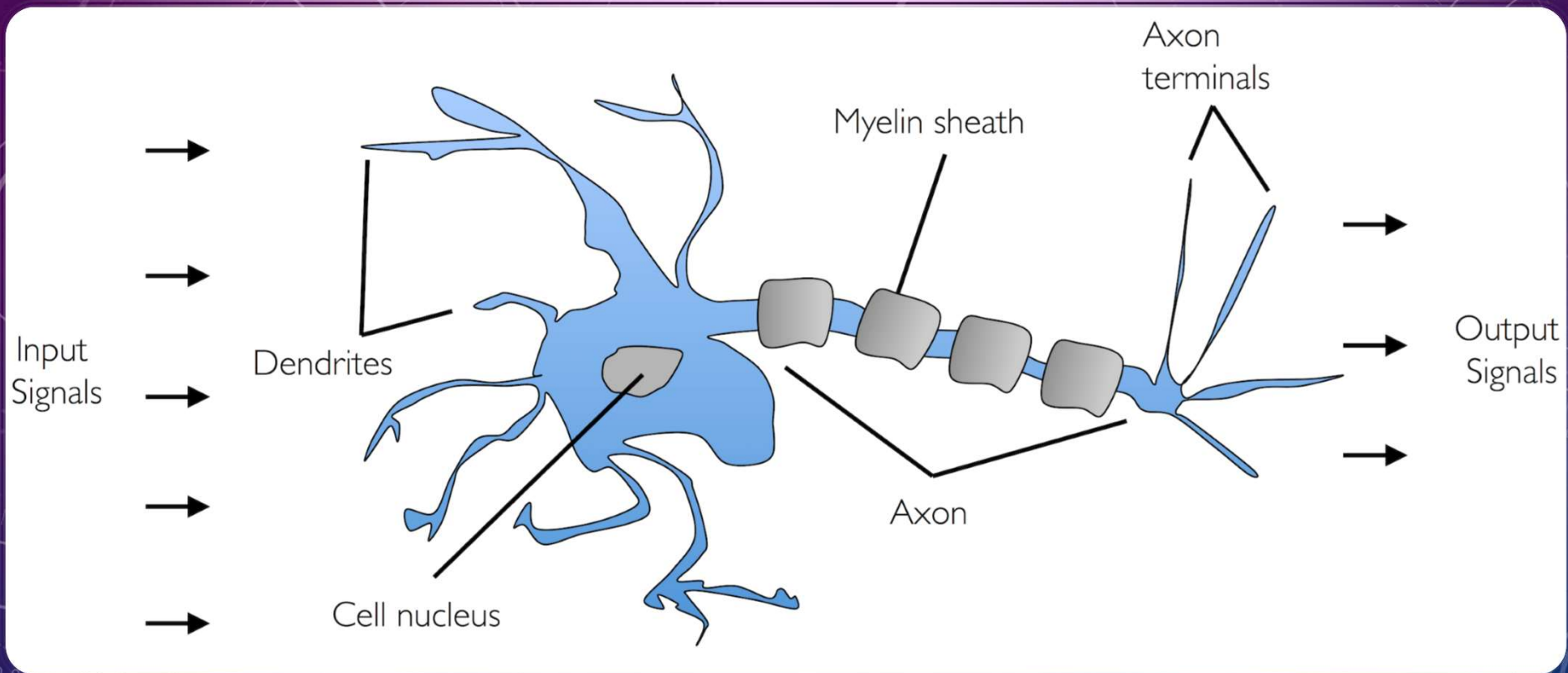
2025 Semester 1

01211373 MACHINE LEARNING AND PROGRAMMING FOR INDUSTRY

DR. VARODOM TOOCHINDA
EMME, KASSETSART UNIVERSITY

LECTURE 2

Simple ML algorithms for classification



NEURON IN BIOLOGICAL BRAIN

MCP NEURON MODEL

(also called perceptron)

$$z = w_1x_1 + w_2x_2 + \dots + w_mx_m:$$

$$\mathbf{w} = \begin{bmatrix} w_1 \\ \vdots \\ w_m \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_m \end{bmatrix}$$

$$\sigma(z) = \begin{cases} 1 & \text{if } z \geq \theta \\ 0 & \text{otherwise} \end{cases}$$

MCP = McCulloch-Pitts

MCP NEURON MODEL

To simplify the code implementation later, we can modify this setup via a couple of steps. First, we move the threshold, θ , to the left side of the equation:

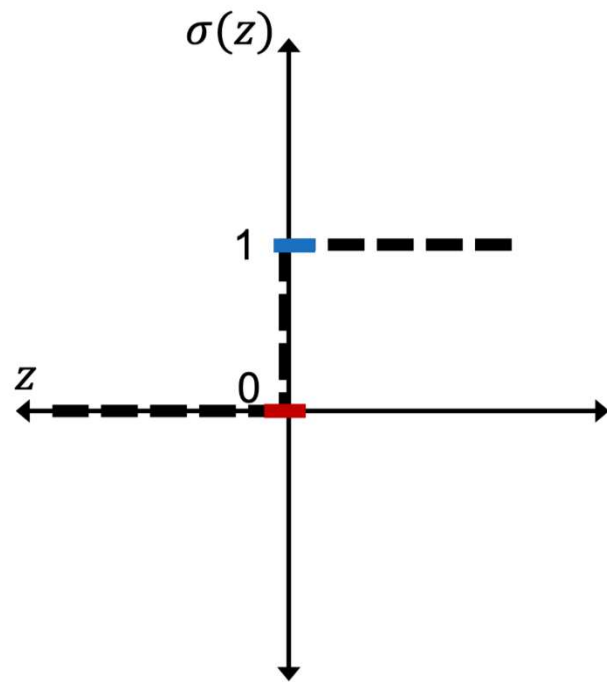
$$\begin{aligned} z &\geq \theta \\ z - \theta &\geq 0 \end{aligned}$$

Second, we define a so-called *bias unit* as $b = -\theta$ and make it part of the net input:

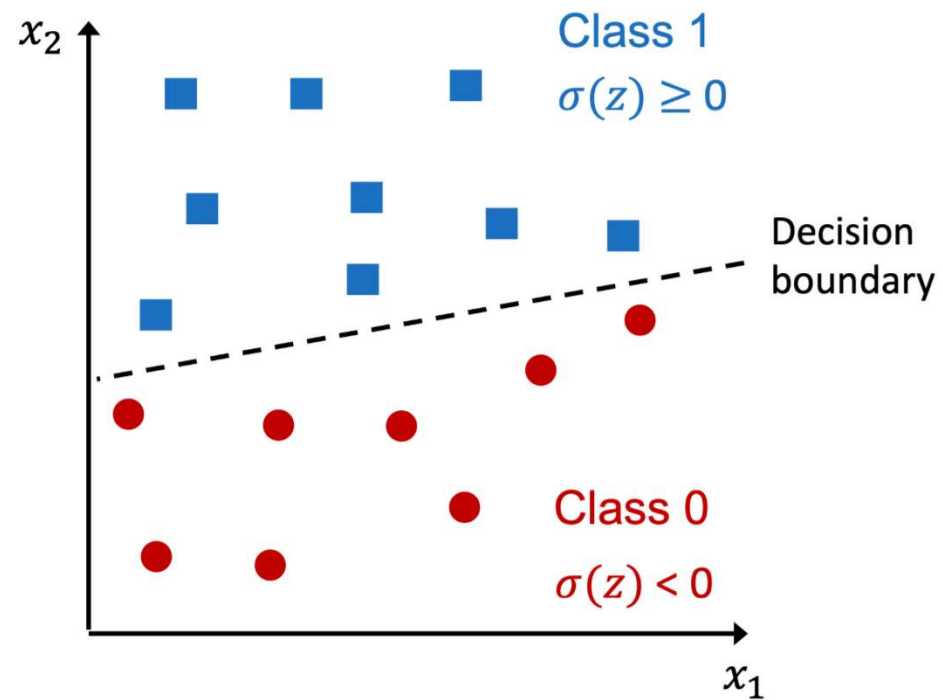
$$z = w_1x_1 + \dots + w_mx_m + b = \mathbf{w}^T\mathbf{x} + b$$

Third, given the introduction of the bias unit and the redefinition of the net input z above, we can redefine the decision function as follows:

$$\sigma(z) = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{otherwise} \end{cases}$$



where $z = \mathbf{w}^T \mathbf{x} + b$



BINARY CLASSIFICATION WITH MCP

PERCEPTRON LEARNING RULE

Frank Rosenblatt (1958)

1. Initialize the weights and bias unit to 0 or small random numbers
2. For each training example, $\mathbf{x}^{(i)}$:
 - a. Compute the output value, $\hat{y}^{(i)}$
 - b. Update the weights and bias unit

Here, the output value is the class label predicted by the unit step function that we defined earlier, and the simultaneous update of the bias unit and each weight, w_j , in the weight vector, $\mathbf{w}$, can be more formally written as:

$$w_j := w_j + \Delta w_j$$

and $b := b + \Delta b$

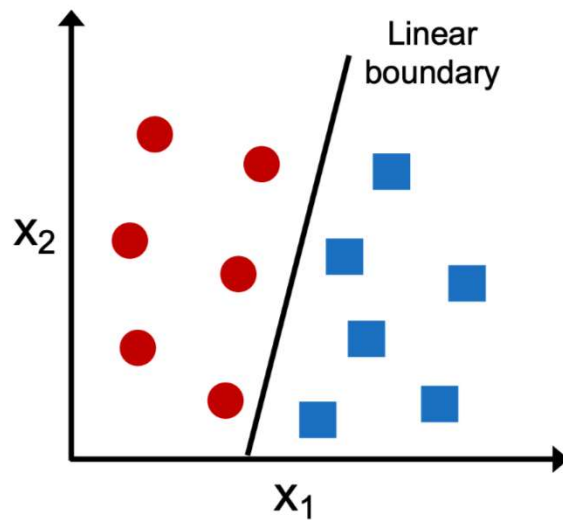
The update values (“deltas”) are computed as follows:

$$\Delta w_j = \eta(y^{(i)} - \hat{y}^{(i)})x_j^{(i)}$$

and $\Delta b = \eta(y^{(i)} - \hat{y}^{(i)})$

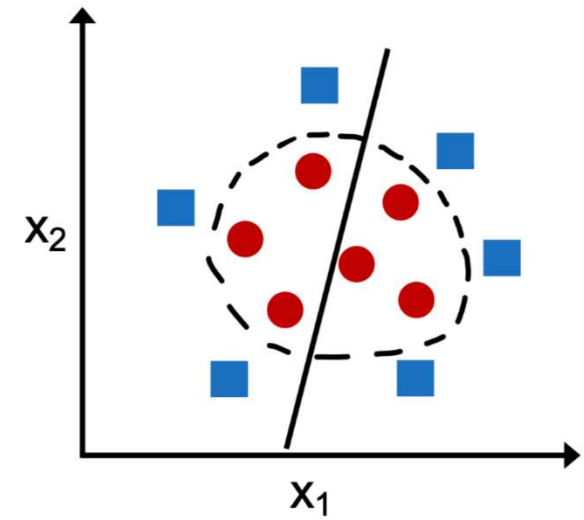
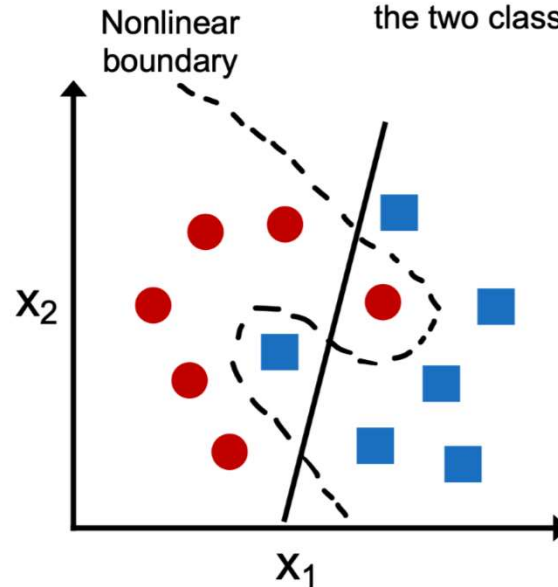
Linearly separable

A linear decision boundary that separates the two classes exists

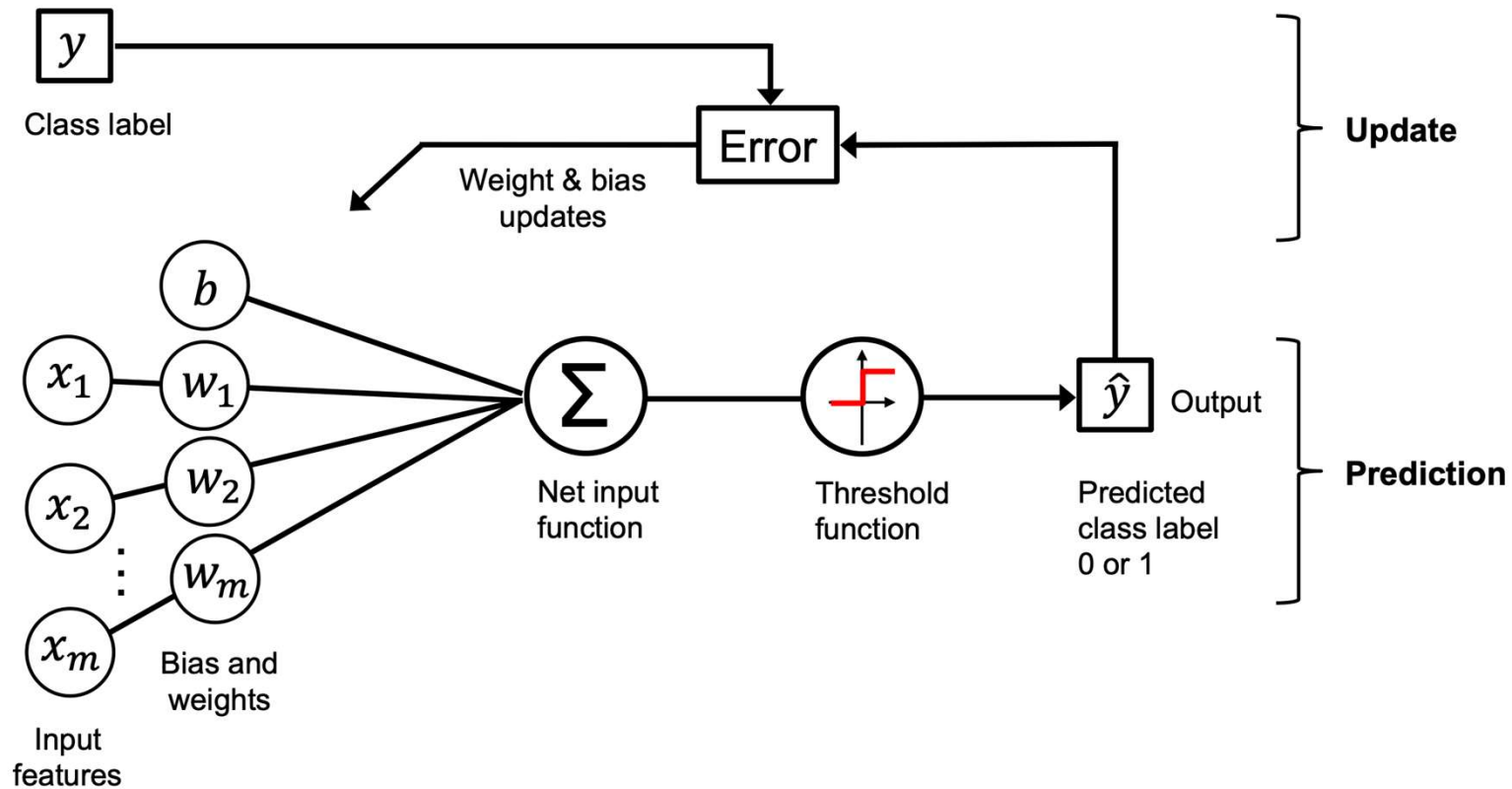


Not linearly separable

No linear decision boundary that separates the two classes perfectly exists



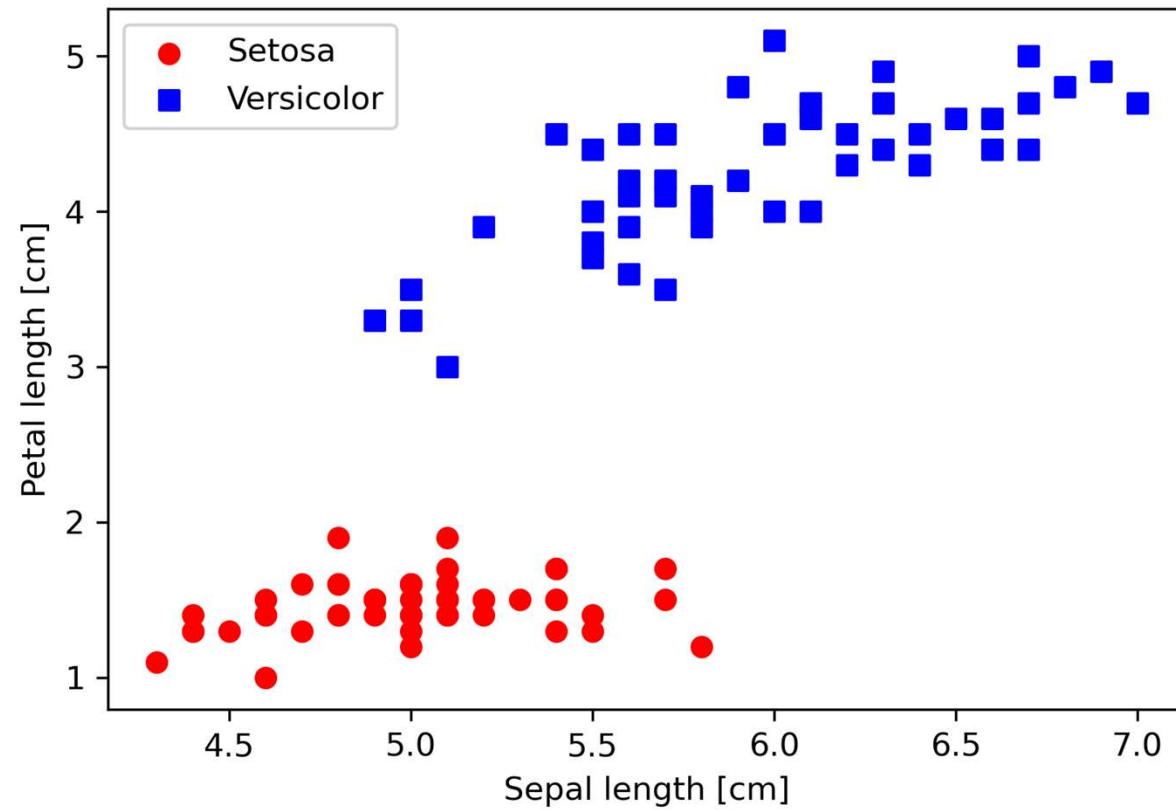
LINEARLY/NON-LINEARLY SEPARABLE CLASSES



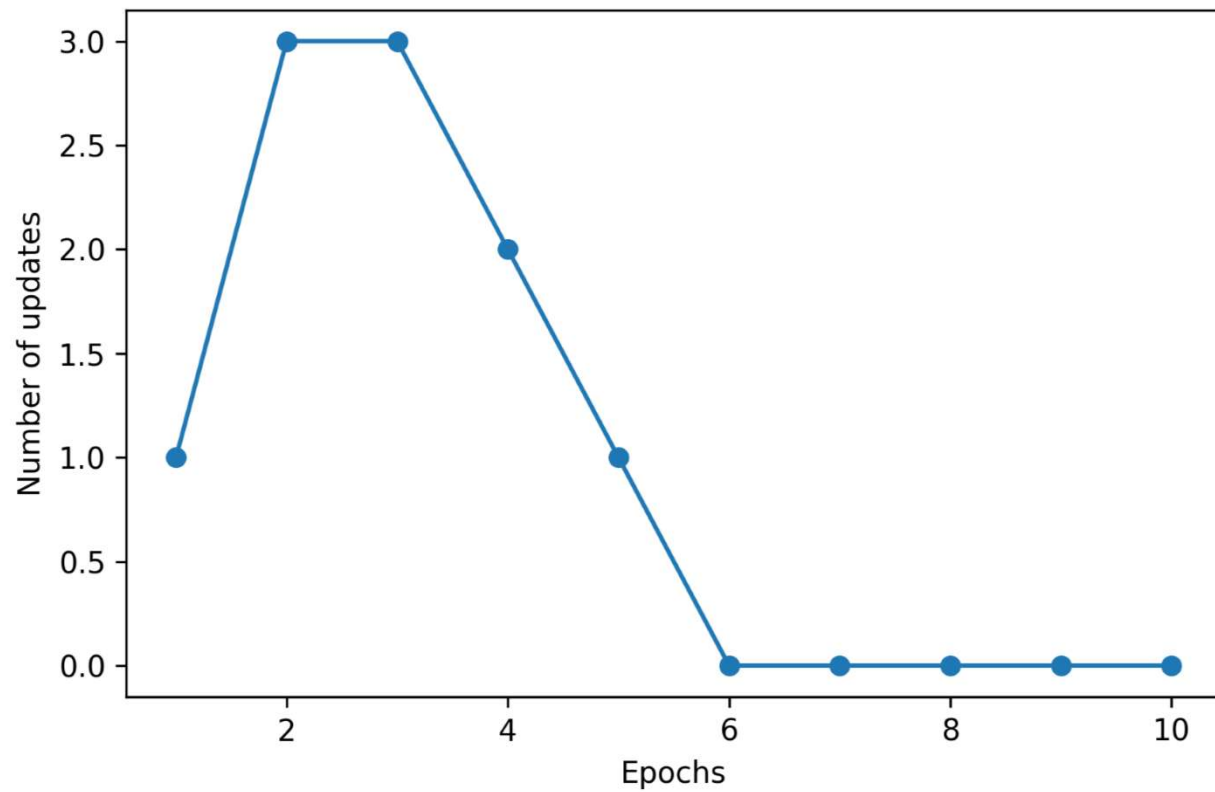
UPDATE WEIGHTS/BIAS BASED ON ERROR

The background is a dark blue to purple gradient. It features several faint, semi-transparent circular patterns and gear-like shapes. A large, complex gear structure is centered in the image. To the left, there are circular scales with numerical markings ranging from 140 to 260. The overall aesthetic is technical and mechanical.

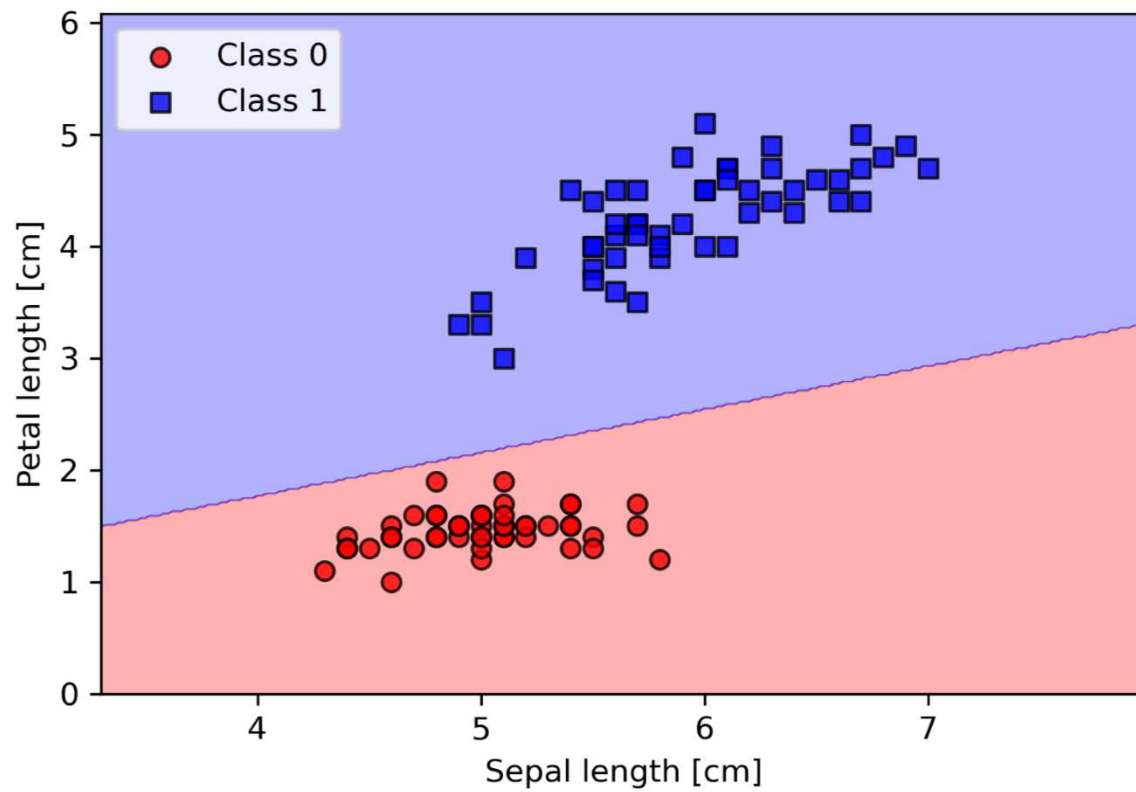
EXERCISE 2.1 : IMPLEMENT OBJECT-ORIENTED PERCEPTRON



BINARY CLASSIFICATION OF IRIS DATASET



MISCLASSIFICATION
ERRORS V.S. EPOCHS



PERCEPTRON'S
DECISION
REGION

PERCEPTRON CONVERGENCE

Two classes must
be separated by a
linear hyperplane

Otherwise, the
weights will never
stop updating

VIDEOS ON PERCEPTRON

- Stanford CS 229 Lecture 4 : <https://youtu.be/iZTeva0WSTQ?si=7YwqTlvXUIIdSD2EY>
- MIT 6.036 Lecture 2 : https://youtu.be/A3obZhw7Jc4?si=u_PfZ1CpZtdQQpaz
- Search “perceptron” and you can find a few interesting videos such as https://youtu.be/l-9ALe3U-Fg?si=_jTaAt69DkInauGJ

ADAPTIVE LINEAR NEURONS (ADALINE)

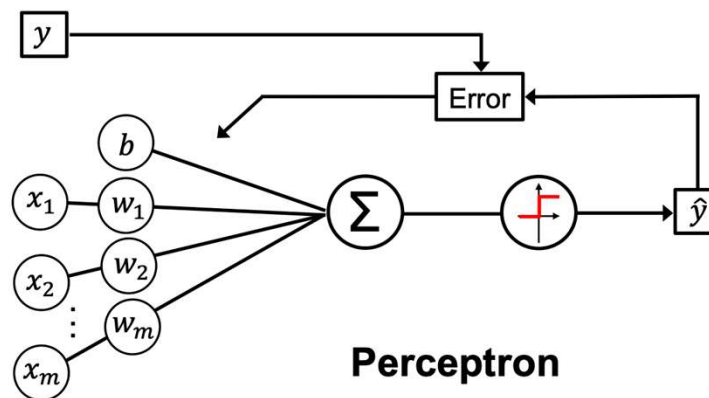
Published by Bernard Widrow and Tedd Hoff
(1960)

Define and minimize continuous loss functions

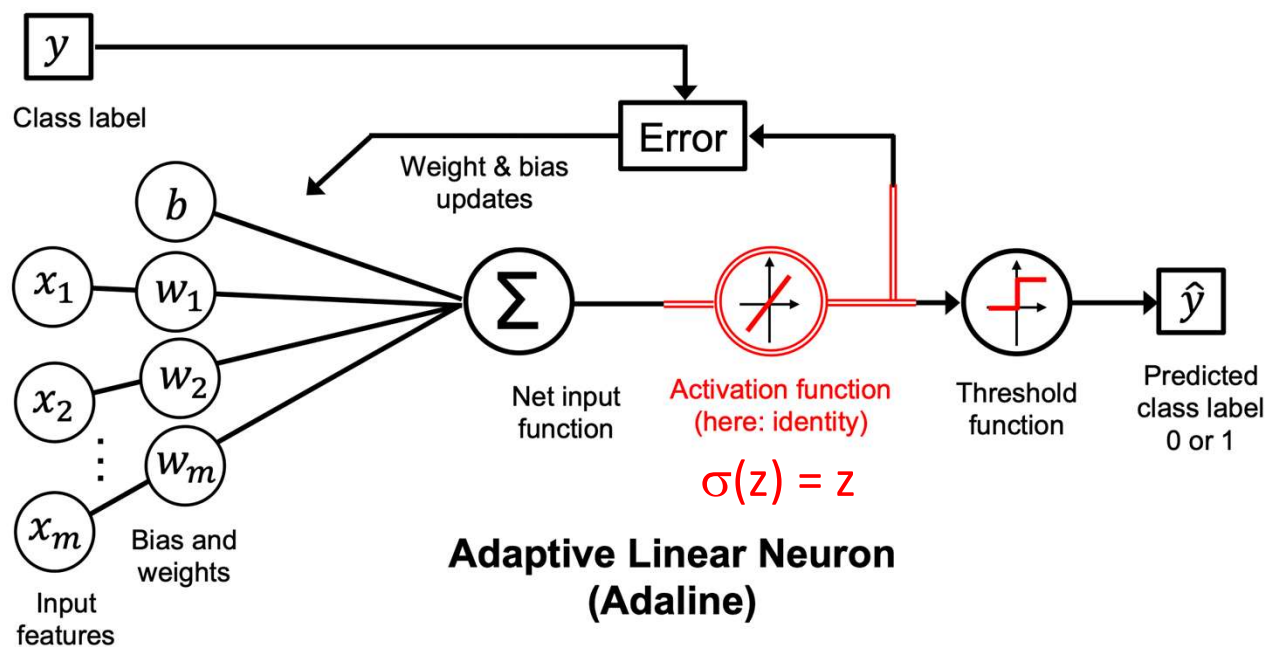
Groundwork for other ML algorithms
(discussed later)

- logistic regression
- linear regression
- support vector machines
- multilayer neural networks

PERCEPTRON V.S. ADALINE



Perceptron



Adaptive Linear Neuron
(Adaline)

Minimizing loss functions with gradient descent

$$L(\mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^n \left(y^{(i)} - \sigma(z^{(i)}) \right)^2$$

MSE loss

$$\mathbf{w} := \mathbf{w} + \Delta \mathbf{w}, \quad b := b + \Delta b$$

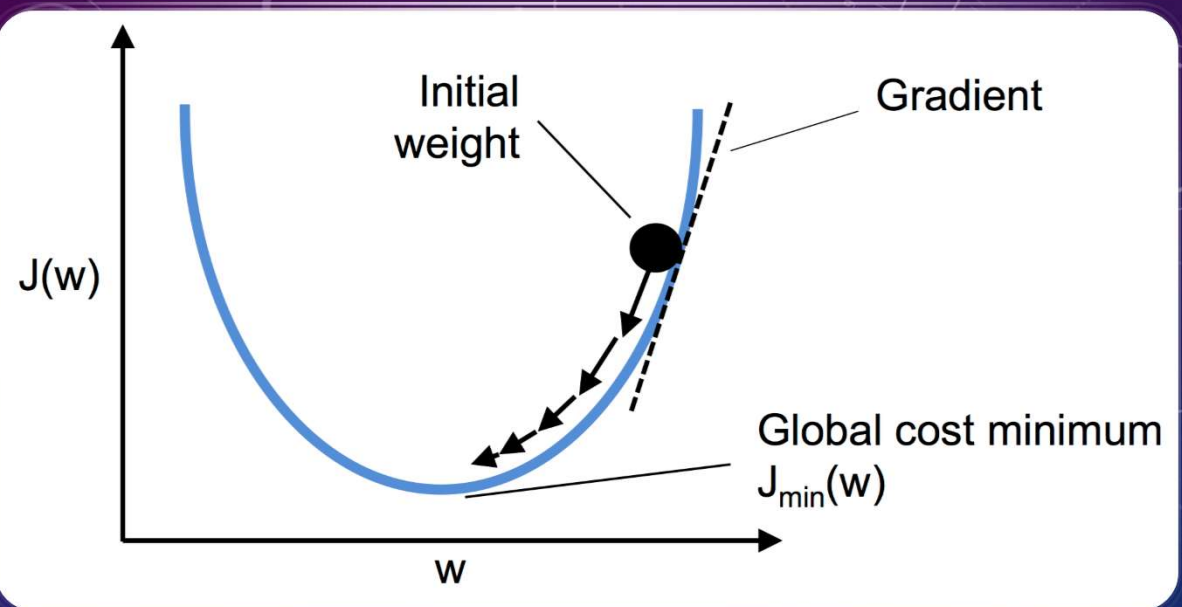
weight and bias update

learning rate

$$\Delta \mathbf{w} = -\eta \nabla_{\mathbf{w}} L(\mathbf{w}, b), \quad \Delta b = -\eta \nabla_b L(\mathbf{w}, b)$$

gradients

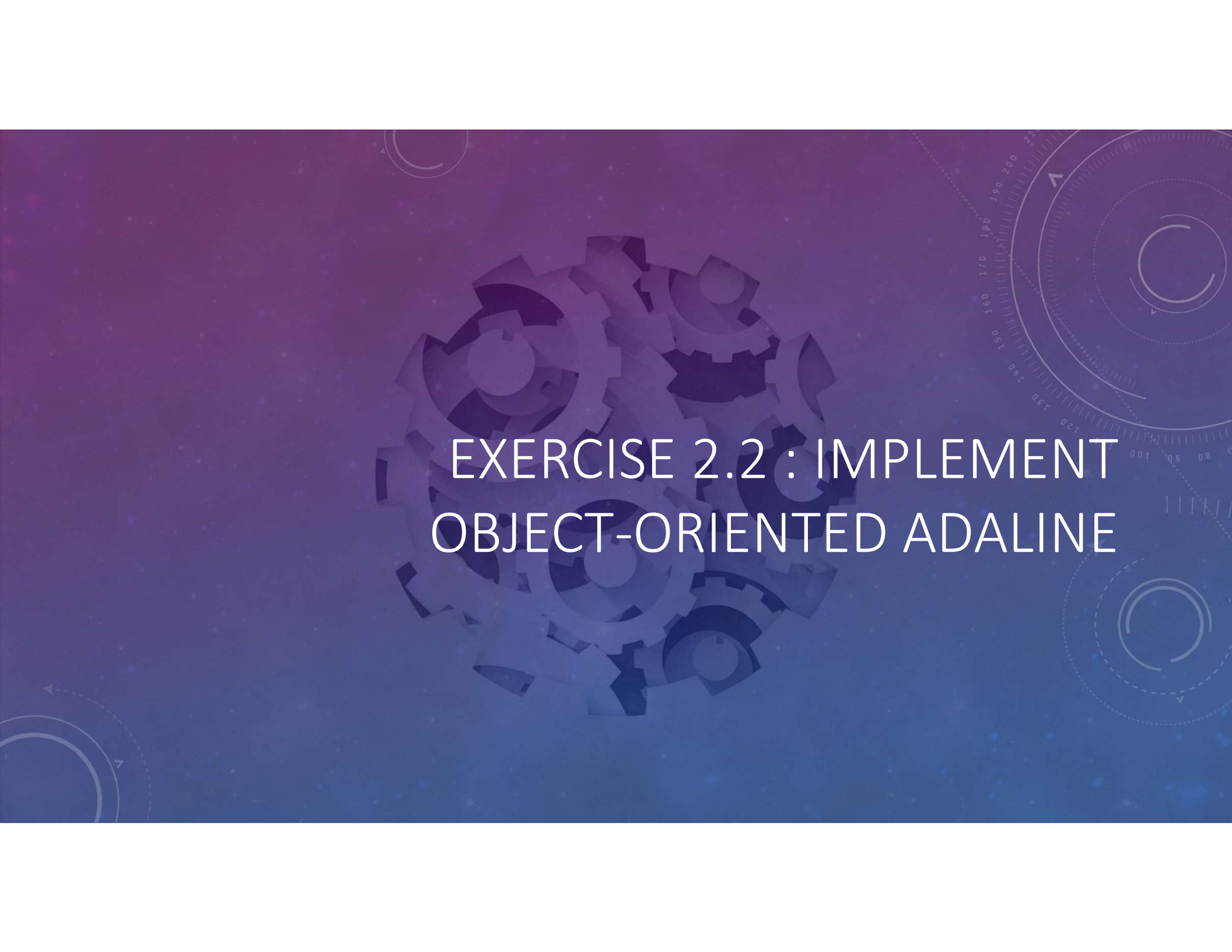
$$\Delta w_j = -\eta \frac{\partial L}{\partial w_j} \quad \text{and} \quad \Delta b = -\eta \frac{\partial L}{\partial b}$$



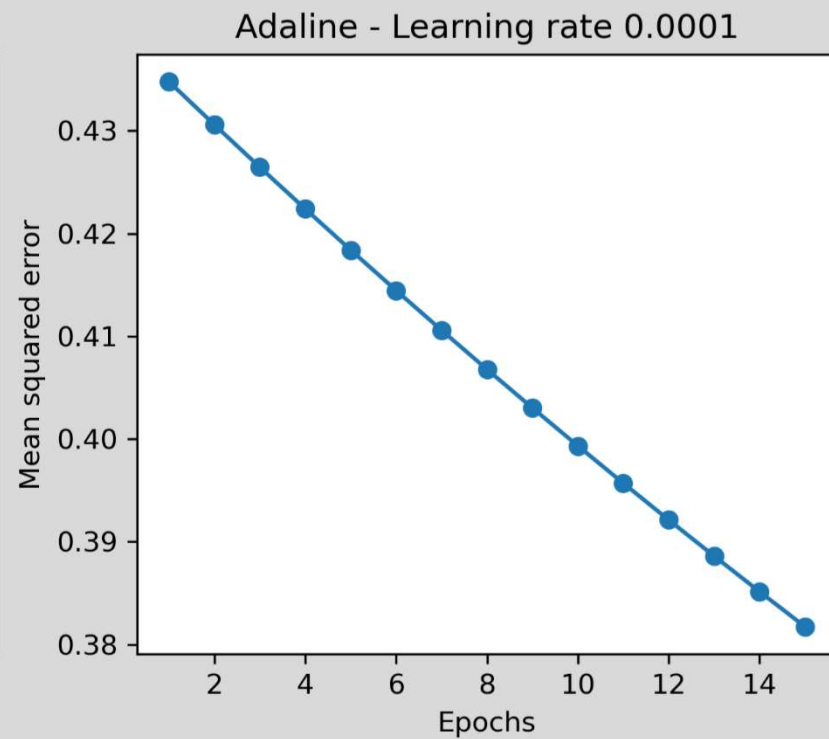
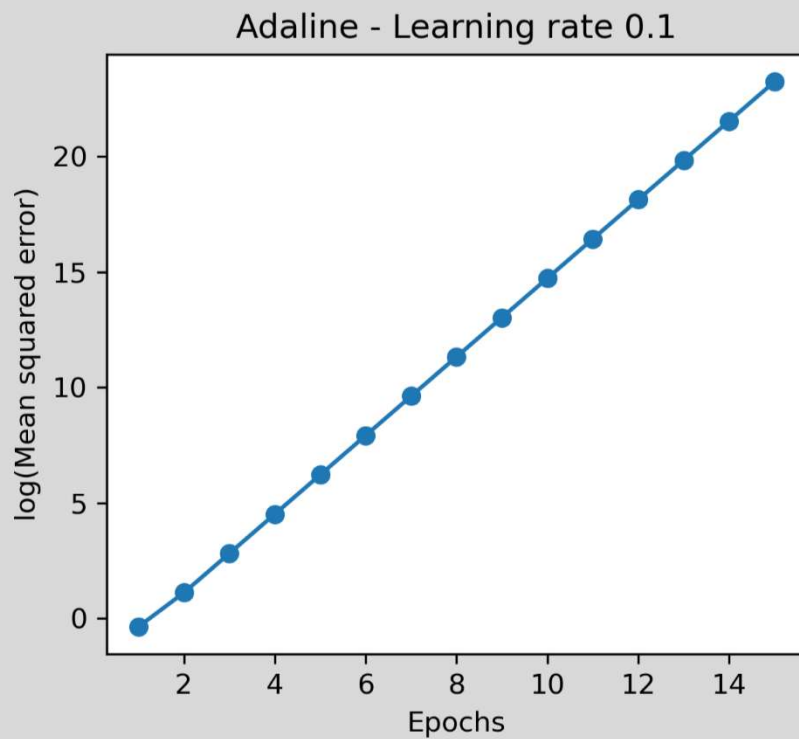
$$\frac{\partial L}{\partial w_j} = -\frac{2}{n} \sum_i \left(y^{(i)} - \sigma(z^{(i)}) \right) x_j^{(i)}$$

details in book

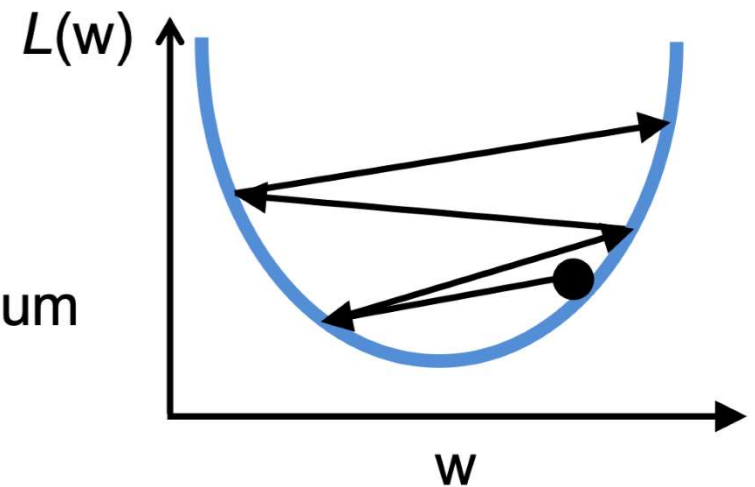
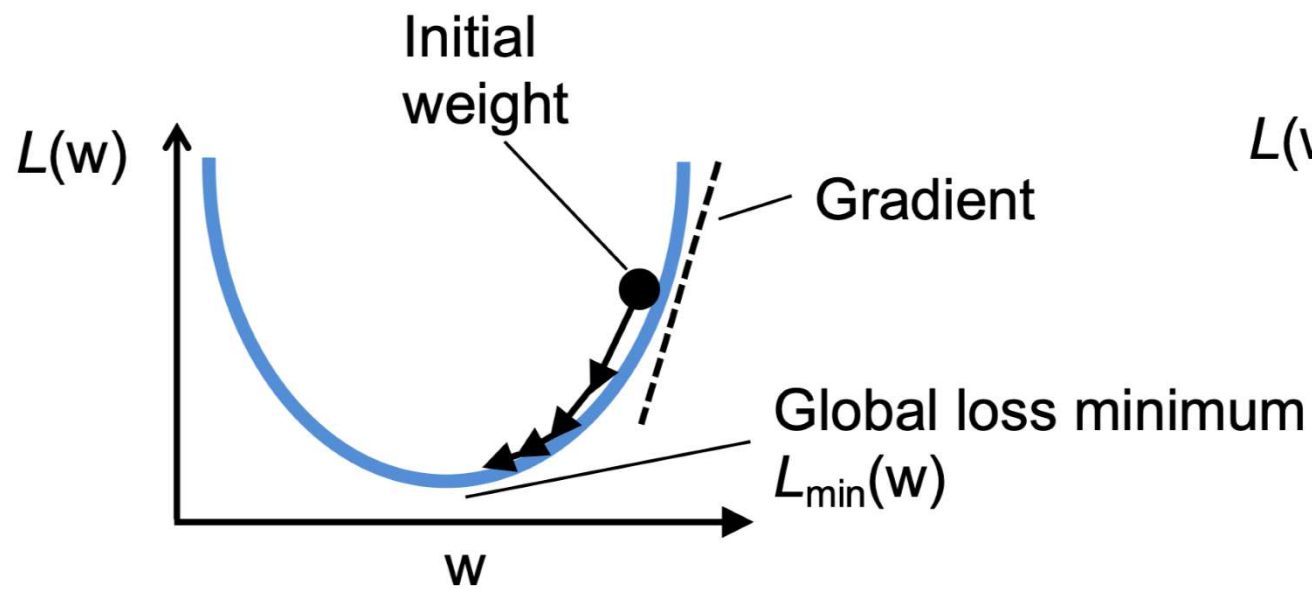
$$\frac{\partial L}{\partial b} = -\frac{2}{n} \sum_i \left(y^{(i)} - \sigma(z^{(i)}) \right)$$

The background is a dark blue to purple gradient. It features several faint, stylized technical drawings. In the center, there is a cluster of interlocking gears. To the right, there is a circular gauge or dial with numerical markings from 0 to 210. In the bottom left, there are some curved lines and arrows, possibly representing a flow or a path. The overall aesthetic is technical and modern.

EXERCISE 2.2 : IMPLEMENT OBJECT-ORIENTED ADALINE

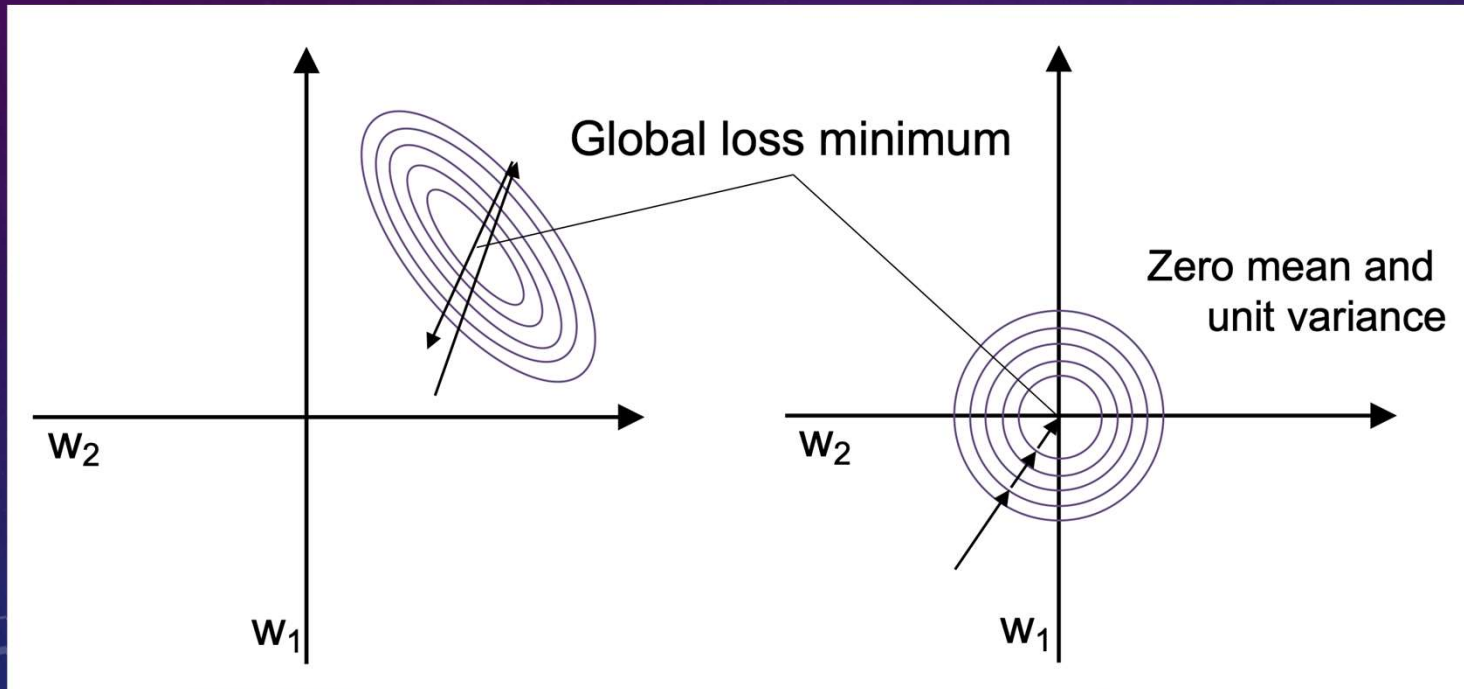


ERROR PLOTS FOR 2 LEARNING RATES



EFFECT OF CHOOSING TOO LARGE LEARNING RATE

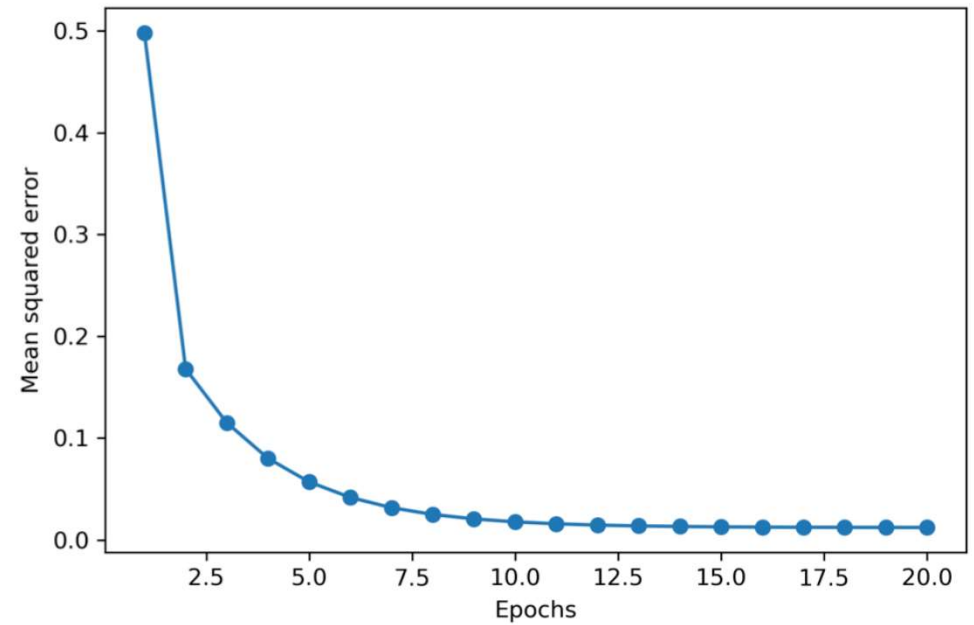
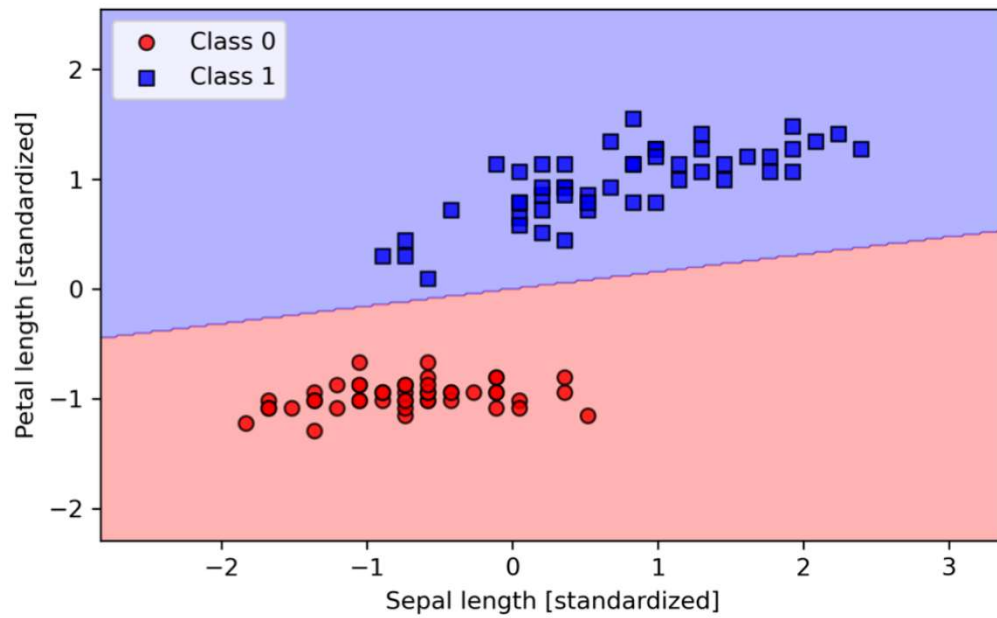
IMPROVE GRADIENT DESCENT WITH FEATURE SCALING (STANDARDIZATION)



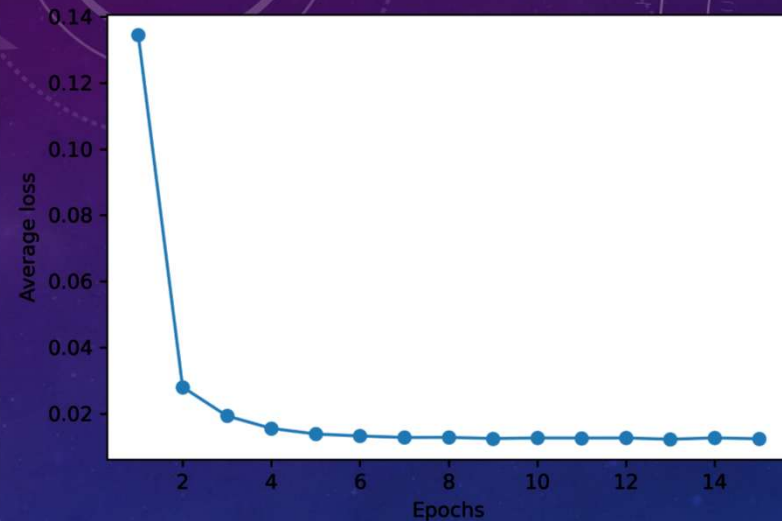
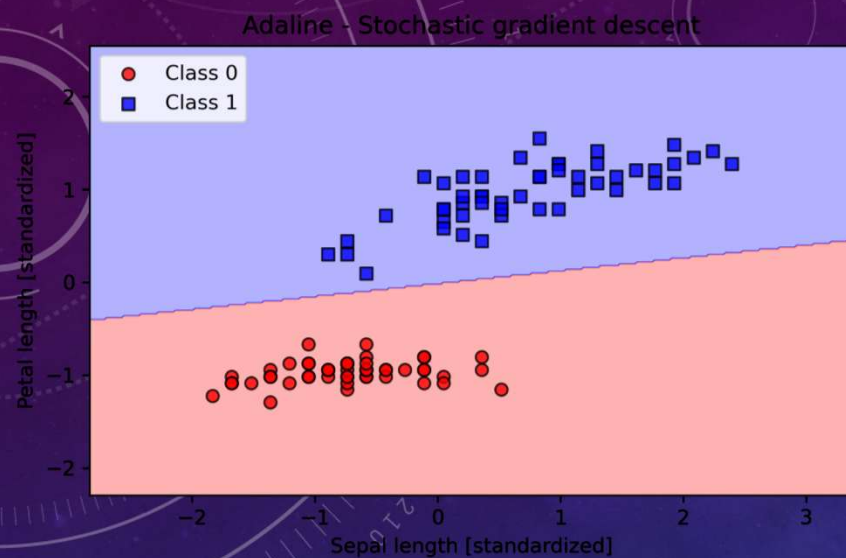
standardize the j th feature
of all training examples

$$x'_j = \frac{x_j - \mu_j}{\sigma_j}$$

Adaline - Gradient descent



DECISION REGION AND MSE



$$\Delta w_j = \eta \left(y^{(i)} - \sigma(z^{(i)}) \right) x_j^{(i)}, \quad \Delta b = \eta \left(y^{(i)} - \sigma(z^{(i)}) \right)$$

STOCHASTIC GRADIENT DESCENT

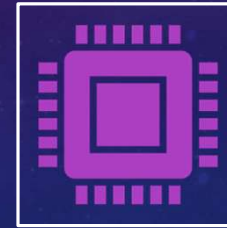
REFERENCES



1. S. Raschka, Y. Liu and V. Mirjalili. Machine Learning with PyTorch and Scikit-Learn. Packt Publishing. 2022.
<https://sebastianraschka.com/blog/2022/ml-pytorch-book.html>



2. A. Ng. CS229 Lecture Notes. Stanford University. 2023.
https://cs229.stanford.edu/main_notes.pdf



3. T. Broderick. Introduction to Machine Learning course (6.036). Massachusetts Institute of Technology. 2020.
<https://tamarabroderick.com/ml.html>

* Most of the figures are from [1]